بِسْمِ اللَّهِ الرَّحْمَٰنِ الرَّحِيمِ

École Marocaine des Sciences de l'Ingénieur – EMSI Casablanca

Filière Génie Informatique – Spécialité Intelligence Artificielle & Data

---

Projet de Fin d'Année réalisé au sein de

Bourse de Casablanca – École de la Bourse

Rapport de Projet de Fin d'Année

EduBourse Assistant

Conception et développement d'un assistant conversationnel multilingue multi-agents pour l'éducation financière

Présenté par

NIEMA

Encadré par

Encadrant académique (EMSI) : *[Nom de l'encadrant académique]*

Encadrant professionnel : *[Nom de l'encadrant – École de la Bourse]*

Devant le jury composé de :

Président : *[Nom du président de jury]*

Examinateur : *[Nom de l'examinateur]*

Encadrant : *[Nom de l'encadrant académique]*

Année universitaire 2025 – 2026

À toutes les personnes qui, par leur exigence et leur bienveillance,
m'ont permis d'avancer sur ce projet.

Remerciements

Je tiens à exprimer ma sincère reconnaissance à l'ensemble des personnes qui ont contribué, de près ou de loin, à la réalisation de ce Projet de Fin d'Année.

Mes remerciements s'adressent tout d'abord à mon encadrant professionnel au sein de la Bourse de Casablanca – École de la Bourse, pour son accueil, sa disponibilité, ses conseils avisés et la confiance qu'il m'a accordée tout au long de ce stage. Son expertise du domaine de l'éducation financière et sa connaissance fine des besoins pédagogiques de l'École ont été déterminantes pour orienter les choix fonctionnels du projet.

Je remercie également mon encadrant académique à l'EMSI, pour son suivi rigoureux, sa disponibilité et ses retours constructifs à chaque étape du projet, qui m'ont permis de structurer ma démarche et de progresser avec méthode.

Ma gratitude va aussi à l'ensemble du corps professoral de la filière Génie Informatique, spécialité Intelligence Artificielle & Data de l'EMSI Casablanca, pour la formation de qualité dispensée durant ce cursus, qui m'a donné les bases techniques et méthodologiques nécessaires à la conduite de ce projet.

Enfin, je remercie les membres du jury pour l'intérêt qu'ils portent à ce travail en acceptant de l'évaluer, ainsi que toutes les personnes de mon entourage qui m'ont soutenu durant cette période.

[Ce texte est une base à personnaliser avec les noms exacts de vos encadrants et toute personne que vous souhaitez remercier.]

Résumé

Ce rapport présente le travail réalisé dans le cadre du Projet de Fin d'Année effectué au sein de la Bourse de Casablanca, au profit de l'École de la Bourse. Le projet, intitulé **EduBourse Assistant**, consiste en la conception et le développement d'un assistant conversationnel intelligent, multilingue et multi-agents, destiné à accompagner le grand public dans l'apprentissage des concepts boursiers et financiers.

Le système repose sur une architecture en microservice interrogée via une API REST, orchestrée par un graphe d'agents **LangGraph** qui route chaque requête utilisateur vers l'un des dix workflows métier identifiés (pédagogie financière, formations e-learning, certification CISI, quiz, championnat de la Bourse, orientation, support école, sécurité, hors-domaine, secours). Les réponses à caractère pédagogique sont générées par un pipeline de **génération augmentée par récupération** (RAG) s'appuyant sur une base vectorielle **ChromaDB** indexée avec le modèle d'embedding **BGE-M3**, garantissant des réponses ancrées dans un corpus documentaire validé et systématiquement accompagnées de citations vérifiables.

Une attention particulière a été portée au traitement de quatre langues métier – français, anglais, arabe standard et darija marocaine (en graphie latine et arabe) – ainsi qu'à la sécurité fonctionnelle du système, qui interdit strictement toute recommandation d'investissement personnalisée au profit d'un contenu strictement éducatif. Le projet intègre également une interface vocale (reconnaissance et synthèse via Gemini), une mémoire conversationnelle par fil de discussion, une adaptation du niveau de réponse au profil de l'apprenant, ainsi qu'une intégration avec la plateforme d'automatisation **n8n** via un serveur **MCP** authentifié.

Le système a été validé par une suite de plus de 1 180 tests automatisés côté backend et 14 tests côté MCP, tous passants, ainsi que par une matrice de validation multilingue et sécuritaire couvrant l'ensemble des langues et canaux supportés.

Mots-clés : assistant conversationnel, multi-agents, LangGraph, RAG, ChromaDB, traitement multilingue, darija, FastAPI, éducation financière, Bourse de Casablanca.

Abstract

This report presents the work carried out during the End-of-Year Project completed at Casablanca Stock Exchange (Bourse de Casablanca), for its École de la Bourse. The project, named **EduBourse Assistant**, consists of designing and developing an intelligent, multilingual, multi-agent conversational assistant intended to support the general public in learning stock-market and financial concepts.

The system is built as a microservice queried through a REST API and orchestrated by a **LangGraph** multi-agent graph that routes each user request to one of ten identified business workflows (financial education, e-learning training, CISI certification, quizzes, the Bourse championship, orientation, school support, security, out-of-domain, and fallback). Educational answers are produced by a **Retrieval-Augmented Generation** (RAG) pipeline backed by a **ChromaDB** vector store indexed with the **BGE-M3** embedding model, ensuring that answers remain grounded in an approved knowledge corpus and are systematically accompanied by verifiable citations.

Particular attention was given to processing four business languages – French, English, Modern Standard Arabic and Moroccan Darija (Latin and Arabic script) – as well as to the functional safety of the system, which strictly forbids personalized investment recommendations in favour of purely educational content. The project also integrates a voice interface (speech-to-text and text-to-speech via Gemini), thread-level conversational memory, learner-profile adaptation, and an integration with the **n8n** automation platform through an authenticated **MCP** server.

The system was validated through an automated test suite of more than 1,180 backend tests and 14 MCP tests, all passing, together with a multilingual and security validation matrix covering all supported languages and channels.

Keywords : conversational assistant, multi-agent systems, LangGraph, RAG, ChromaDB, multilingual NLP, Darija, FastAPI, financial education, Casablanca Stock Exchange.

ملخص

يقدّم هذا التقرير العمل المنجَز في إطار مشروع نهاية السنة الذي أُنجز لدى بورصة الدار البيضاء، لفائدة مدرسة البورصة. يتمثّل المشروع، المسمّى **Assistant EduBourse**، في تصميم وتطوير مساعد محادثة ذكي، متعدّد اللغات ومتعدّد الوكلاء، يهدف إلى مواكبة عموم المستخدمين في تعلّم مفاهيم البورصة والتمويل.

يعتمد النظام على معمارية خدمة مصغّرة (microservice) تُستدعى عبر واجهة برمجية REST، ويُنسَّق عملها بواسطة رسم بياني متعدّد الوكلاء **LangGraph** يوجّه كل طلب من المستخدم إلى أحد المسارات الوظيفية العشرة المحدَّدة. تُولَّد الإجابات ذات الطابع التربوي عبر خط أنابيب **RAG** (التوليد المعزَّز بالاسترجاع) يعتمد على قاعدة بيانات متجهية **ChromaDB**، مما يضمن إجابات مرتكزة على مدوّنة موثَّقة ومرفقة دائماً بمراجع قابلة للتحقّق.

أُولي اهتمام خاص لمعالجة أربع لغات عمل: الفرنسية، الإنجليزية، العربية الفصحى، والدارجة المغربية (بالحرفين اللاتيني والعربي)، وكذا للسلامة الوظيفية للنظام الذي يمنع بشكل صارم أي توصية استثمارية شخصية، لفائدة محتوى تربوي بحت.

الكلمات المفتاحية: مساعد محادثة، نظام متعدّد الوكلاء، LangGraph، RAG، معالجة متعدّدة اللغات، الدارجة، التربية المالية، بورصة الدار البيضاء.

Table des matières

Remerciements **i**

Résumé **ii**

Abstract **iii**

ملخص **iv**

Liste des abréviations **x**

Introduction générale **1**

Contexte et motivation 1
Problématique 2
Objectifs du projet 2
Plan du rapport 3

1 Contexte général du projet **4**

1.1 Présentation de l'organisme d'accueil 4
1.1.1 La Bourse de Casablanca 4
1.1.2 L'École de la Bourse 4
1.2 Contexte et problématique du projet 5
1.3 Objectifs du projet 5
1.4 Démarche méthodologique et planification 6
1.5 Conclusion du chapitre 7

2 Analyse et spécification des besoins **8**

2.1 Étude de l'existant et positionnement 8
2.2 Identification des acteurs 8
2.3 Besoins fonctionnels 9
2.4 Diagramme de cas d'utilisation 10
2.5 Besoins non fonctionnels 10

2.6 Conclusion du chapitre 12

3 Conception **14**
3.1 Architecture générale 14
3.2 Architecture technique en couches 15
3.3 Le graphe multi-agents LangGraph 16
3.3.1 Politique de sécurité 17
3.4 Diagramme de séquence – requête pédagogique ancrée (RAG) 17
3.5 Diagramme de classes simplifié 18
3.6 Choix technologiques et justification 18
3.7 Conclusion du chapitre 20

4 Réalisation **22**
4.1 Environnement de travail 22
4.2 Pipeline de traitement multilingue 22
4.3 Sécurité et conformité 23
4.4 Récupération documentaire et génération ancrée (RAG) 24
4.5 Agents métier spécialisés 25
4.6 Mémoire conversationnelle et personnalisation 26
4.7 Interface vocale 26
4.8 Intégration n8n et serveur MCP 27
4.9 Interface de démonstration (frontend) 27
4.10 Conclusion du chapitre 27

5 Tests, validation et résultats **28**
5.1 Stratégie de tests 28
5.2 Résultats de l'audit technique final 28
5.3 Validation du comportement en situation d'échec 29
5.4 Limitations connues 29
5.5 Scénario de démonstration 30
5.6 Conclusion du chapitre 31

Conclusion générale et perspectives **32**
Perspectives d'évolution 32

Bibliographie et webographie **34**

A Exemple de requête et de réponse de l'API **35**

B Variables de configuration principales **36**

C Statuts canoniques de génération ancrée **37**

D Journal des jalons de développement **38**

Table des figures

2.1 Diagramme de cas d'utilisation simplifié d'EduBourse Assistant 11

3.1 Architecture générale et parcours des requêtes texte, vocale et automatisée 15
3.2 Diagramme de séquence – traitement d'une requête pédagogique ancrée . 18
3.3 Diagramme de classes simplifié – objets d'échange API / graphe / RAG . 21

Liste des tableaux

1.1 Grandes phases du projet et livrables associés 6

2.1 Workflows métier (routes) prises en charge par le système 9
2.2 Besoins non fonctionnels 12

3.1 Principaux modules du backend 15
3.2 Choix technologiques et justification 18

4.1 Comportement des agents métier 25

5.1 Résultats de la validation finale 29
5.2 Limitations connues documentées à l'issue du projet 29

Liste des abréviations

Sigle	Signification
API	Application Programming Interface (interface de programmation)
ARY	Code langue business désignant la darija marocaine (*Moroccan Arabic*)
BGE-M3	*BAAI General Embedding – Multi-lingual, Multi-functionality, Multi-granularity*, modèle d'embedding vectoriel
CISI	*Chartered Institute for Securities & Investment*, certification professionnelle en finance
E2E	*End-to-End* (bout en bout)
EMSI	École Marocaine des Sciences de l'Ingénieur
FastAPI	Framework Python pour la construction d'API web asynchrones
LangGraph	Bibliothèque d'orchestration d'agents sous forme de graphe d'états
LLM	*Large Language Model* (grand modèle de langage)
MCP	*Model Context Protocol*, protocole standard d'exposition d'outils à un agent IA
MSA	*Modern Standard Arabic* (arabe standard moderne)
n8n	Plateforme open-source d'automatisation de workflows
PFA	Projet de Fin d'Année
RAG	*Retrieval-Augmented Generation* (génération augmentée par récupération)
REST	*Representational State Transfer*, style d'architecture d'API web
STT	*Speech-to-Text* (reconnaissance vocale)
TTS	*Text-to-Speech* (synthèse vocale)
UML	*Unified Modeling Language* (langage de modélisation unifié)

Sigle	**Signification**
UI/UX	*User Interface / User Experience*

Introduction générale

L'éducation financière constitue aujourd'hui un enjeu majeur pour le développement des marchés de capitaux et pour la protection de l'épargnant. Au Maroc, la Bourse de Casablanca a fait de la démocratisation de la culture boursière l'un des axes stratégiques de son action, notamment à travers l'École de la Bourse, structure dédiée à la formation du grand public aux mécanismes des marchés financiers.

Dans un contexte marqué par la multiplication des canaux numériques et par une demande croissante d'accompagnement personnalisé et disponible à tout moment, l'École de la Bourse a souhaité explorer la mise en place d'un assistant conversationnel intelligent capable de répondre, de manière fiable, pédagogique et sécurisée, aux questions du public sur les notions boursières, les formations disponibles et les services associés. C'est dans ce cadre que s'inscrit le présent Projet de Fin d'Année, intitulé **EduBourse Assistant**.

Contexte et motivation

Les assistants conversationnels fondés sur les grands modèles de langage (LLM) ouvrent des perspectives nouvelles pour l'éducation financière : disponibilité continue, capacité à traiter le langage naturel dans plusieurs langues, et personnalisation du contenu selon le niveau de l'utilisateur. Ces mêmes technologies posent cependant des risques spécifiques dans un contexte financier : réponses non vérifiées (« hallucinations »), dérive vers un conseil en investissement non autorisé, ou encore incapacité à traiter correctement les langues locales telles que la darija marocaine.

Le projet EduBourse Assistant a été conçu pour répondre à ces deux exigences simultanément : offrir une expérience conversationnelle naturelle et multilingue, tout en garantissant que chaque réponse à caractère pédagogique reste ancrée dans un corpus de connaissances validé et qu'aucune recommandation d'investissement personnalisée ne puisse être produite.

Problématique

La problématique centrale de ce projet peut être formulée ainsi : *comment concevoir un assistant conversationnel capable de traiter des requêtes en français, en anglais, en arabe standard et en darija marocaine, de router chaque demande vers le bon service métier de l'École de la Bourse, de générer des réponses pédagogiques fiables et vérifiables, tout en garantissant un haut niveau de sécurité fonctionnelle et une architecture réutilisable par les canaux existants (site web, plateforme vocale, automatisation) ?*

Cette problématique se décline en plusieurs sous-questions techniques traitées tout au long de ce rapport :

- Quelle architecture logicielle permet d'orchestrer plusieurs agents spécialisés tout en gardant un point de contrôle unique pour la sécurité et le routage ?
- Comment garantir que les réponses à caractère financier restent ancrées dans un corpus documentaire approuvé plutôt que générées librement par un modèle de langage ?
- Comment détecter et traiter correctement quatre langues métier, dont une langue à faible ressource (la darija), y compris dans ses variantes graphiques latine et arabe ?
- Comment intégrer le système à des canaux externes (vocal, automatisation n8n) sans dupliquer la logique métier ni affaiblir la sécurité ?

Objectifs du projet

Les objectifs assignés à ce PFA sont les suivants :

- concevoir une architecture multi-agents orchestrée par un graphe d'états (LangGraph), avec un routage métier explicite vers dix workflows identifiés ;
- mettre en place un pipeline de génération augmentée par récupération (RAG) sur une base documentaire vectorisée, avec validation stricte des citations et des évidences ;
- développer un module de traitement multilingue couvrant le français, l'anglais, l'arabe standard et la darija marocaine (graphies latine et arabe) ;
- implémenter une politique de sécurité fonctionnelle interdisant toute recommandation d'investissement personnalisée ;
- fournir un canal vocal (reconnaissance et synthèse de la parole) et une intégration à la plateforme d'automatisation n8n via un serveur MCP authentifié ;

- développer une interface de démonstration permettant de visualiser le fonctionnement de chacun des sous-systèmes ;
- valider l'ensemble du système par une suite de tests automatisés et un scénario de démonstration couvrant les quatre langues et les principaux workflows.

Plan du rapport

Ce rapport est organisé en cinq chapitres. Le premier chapitre présente le contexte général du projet : l'organisme d'accueil, la problématique, les objectifs et la démarche méthodologique adoptée. Le deuxième chapitre est consacré à l'analyse et à la spécification des besoins fonctionnels et non fonctionnels. Le troisième chapitre détaille la conception générale et technique du système, avec la modélisation UML associée. Le quatrième chapitre décrit la réalisation des différents modules du système. Le cinquième chapitre présente la stratégie de tests, les résultats de validation obtenus ainsi que les limites connues du système. Le rapport se termine par une conclusion générale ouvrant sur les perspectives d'évolution du projet.

Chapitre 1

Contexte général du projet

Ce premier chapitre présente le cadre général dans lequel s'inscrit le projet : l'organisme d'accueil, la problématique métier, les objectifs poursuivis et la démarche méthodologique suivie pour conduire le projet du cadrage jusqu'à la validation finale.

1.1 Présentation de l'organisme d'accueil

1.1.1 La Bourse de Casablanca

La Bourse de Casablanca est le marché officiel des valeurs mobilières du Royaume du Maroc. Créée en 1929 sous le nom d'*Office de compensation des valeurs mobilières*, elle a connu trois réformes structurantes en 1967, 1986 et 1993, qui l'ont progressivement rapprochée des standards internationaux : dématérialisation des titres, cotation électronique généralisée à partir de 1997 en remplacement de la cotation à la criée, puis migration en 2008 vers le système de négociation NSC V900, également utilisé par Euronext [1].

Elle a aujourd'hui le statut de société anonyme à conseil d'administration, dont le capital est détenu par les principales banques marocaines, la Caisse de Dépôt et de Gestion, les sociétés de bourse et des compagnies d'assurance. Avec une capitalisation de plusieurs centaines de milliards de dirhams, elle est considérée comme l'une des toutes premières places financières du continent africain [1].

1.1.2 L'École de la Bourse

Fondée en 2000, l'École de la Bourse est la structure de la Bourse de Casablanca dédiée à l'éducation financière du grand public [2]. Elle poursuit quatre axes stratégiques :

- promouvoir le marché boursier marocain auprès du grand public ;
- développer l'éducation financière des citoyens ;

- équiper les investisseurs actuels et futurs des outils nécessaires à la compréhension des marchés ;
- élever le niveau de connaissance financière d'un public cible très diversifié (élèves, étudiants, salariés, investisseurs, professions libérales, épargnants, retraités).

L'École propose trois modalités de formation : des sessions en présentiel, des formations en ligne (e-learning), et des formations certifiantes en partenariat avec le **CISI** (*Chartered Institute for Securities & Investment*). Depuis sa création, elle a formé environ 80 000 participants au travers de dispositifs tels que les Journées Bourse, les ateliers investisseurs, la Semaine de la finance pour enfants, ou encore le **Championnat de la Bourse**, compétition nationale dans laquelle les participants gèrent un portefeuille fictif de valeurs boursières [2].

C'est ce périmètre métier – pédagogie financière, formations, certification CISI, quiz, championnat, orientation et support pratique – qui constitue le socle fonctionnel du projet EduBourse Assistant, présenté en détail au chapitre 2.

1.2 Contexte et problématique du projet

Le canal humain de formation de l'École de la Bourse, bien qu'efficace, ne peut répondre en continu et instantanément aux questions simples et récurrentes du public (définitions, orientation, informations pratiques). Un assistant conversationnel disponible en permanence, capable de comprendre et de répondre dans la langue choisie par l'utilisateur – y compris la darija marocaine, très largement utilisée à l'oral comme à l'écrit dans les échanges numériques informels – représente un complément naturel à l'offre existante.

Ce type de système présente toutefois un risque spécifique au domaine financier : un modèle de langage générique, non contraint, peut produire des réponses inexactes ou dériver vers un conseil en investissement personnalisé, ce qui est strictement encadré par la réglementation et proscrit par la politique de l'École. La problématique du projet est donc double : garantir une expérience conversationnelle riche et multilingue, tout en assurant que le système reste strictement pédagogique, vérifiable et sécurisé – problématique détaillée dans l'introduction générale de ce rapport.

1.3 Objectifs du projet

Les objectifs du projet, détaillés dans l'introduction générale, se résument à la conception d'un système multi-agents multilingue, ancré sur un corpus documentaire validé (RAG), doté d'une politique de sécurité stricte, et intégrable aux canaux existants de l'École (site web, vocal, automatisation). Le chapitre 2 détaille la traduction de ces objectifs en

besoins fonctionnels et non fonctionnels.

1.4 Démarche méthodologique et planification

Le projet a été conduit selon une démarche itérative et incrémentale, proche de l'esprit agile, organisée en jalons successifs (notés `J1` à `J9` dans le journal de développement du projet), chaque jalon correspondant à un sous-ensemble cohérent de fonctionnalités livrées, testées et documentées avant de passer au suivant. Cette démarche a permis de sécuriser l'avancement du projet malgré la complexité du système (multi-agents, multilingue, RAG, vocal) en évitant un développement « big bang » difficile à valider.

Le tableau 1.1 synthétise les grandes phases du projet et leurs livrables principaux.

Tab. 1.1 : Grandes phases du projet et livrables associés

Phase	**Objectif**	**Livrables principaux**
Cadrage	Analyse du besoin, étude de l'existant, définition du périmètre métier	Cahier des charges fonctionnel, découpage en 10 workflows métier
Fondations (J1–J2)	Mise en place du socle technique	API FastAPI, schémas de requête/réponse, infrastructure Docker (backend, PostgreSQL, ChromaDB)
Traitement multilingue (J2)	Détection de langue et de script	Détecteurs FR/EN/AR/darija (latin et arabe), validation de script, jeux d'évaluation (FLORES-200, corpus dédiés)
Noyau multi-agents (J3)	Orchestration LangGraph, sécurité, routage	Graphe LangGraph, politique de sécurité, gestion des points de reprise (*checkpoints*), 320 tests
RAG et agents métier (J4)	Génération ancrée, agents spécialisés	Pipeline RAG (ChromaDB + BGE-M3), agents pédagogie/formation/CISI/quiz/-championnat/orientation/support, API v1 complète
Mémoire & personnalisation	Continuité conversationnelle, profils	Mémoire de fil de discussion, profils apprenant (débutant/intermédiaire/avancé)
Canal vocal & intégrations	Extension multicanal	STT/TTS Gemini, intégration n8n, serveur MCP authentifié

Phase	Objectif	Livrables principaux
Frontend & démonstration	Interface de démonstration	Tableau de bord React/TypeScript multi-pages
Validation finale (J9-01 à J9-05)	Audit technique final	1 181 tests backend, 14 tests MCP, matrice de validation sécurité/multilingue, scénario de démonstration, rapport de PFA

Cette organisation par jalons a également structuré la gestion des risques techniques du projet : chaque jalon se terminait par une phase de stabilisation (correction de régressions, mise à jour de la documentation) avant l'ouverture du jalon suivant, ce qui explique la présence, dans le journal de développement, de statuts intermédiaires tels que `STABLE_PROVISIONAL` pour les jalons non encore auditionnés de façon indépendante.

1.5 Conclusion du chapitre

Ce chapitre a présenté le cadre du projet : un organisme d'accueil de premier plan dans l'écosystème financier marocain, une problématique conjuguant richesse conversationnelle et exigence de sécurité, et une démarche méthodologique itérative organisée en jalons. Le chapitre suivant détaille l'analyse des besoins qui a guidé les choix de conception présentés au chapitre 3.

Chapitre 2

Analyse et spécification des besoins

Ce chapitre présente l'analyse fonctionnelle du système : les acteurs identifiés, les besoins fonctionnels traduits en workflows métier, le diagramme de cas d'utilisation associé, et les besoins non fonctionnels qui ont guidé les choix de conception du chapitre 3.

2.1 Étude de l'existant et positionnement

Avant le lancement du projet, l'accompagnement du public de l'École de la Bourse reposait principalement sur des canaux humains (formateurs, conseillers) et sur du contenu statique (site web, supports de formation). Ces canaux, bien qu'adaptés à un accompagnement approfondi, ne permettent pas de répondre de façon instantanée et disponible en continu aux questions récurrentes du public, ni de s'adapter automatiquement à la langue et au niveau de l'utilisateur.

Les assistants conversationnels génériques du marché (chatbots FAQ classiques, assistants LLM généralistes) présentent quant à eux deux limites majeures pour ce contexte : l'absence de garantie d'ancrage des réponses dans un corpus validé (risque de réponses inexactes ou « hallucinées »), et l'absence de politique de sécurité spécifique au domaine financier réglementé. EduBourse Assistant se positionne donc comme un système *métier*, combinant les avantages conversationnels des LLM avec les garanties de traçabilité d'un système à base de règles et de récupération documentaire.

2.2 Identification des acteurs

- **Utilisateur / apprenant** – toute personne du grand public interagissant avec l'assistant, en texte ou en voix, dans l'une des quatre langues métier supportées.
- **Plateforme d'intégration (n8n)** – acteur système représentant les canaux ex-

ternes (site web, widget, automatisation) qui relaient une requête utilisateur vers l'assistant via un webhook authentifié.

- **Serveur MCP** – composant d'interface exposant un jeu d'outils restreint (`get_backend_health` `ask_edubourse`) aux clients autorisés, sans capacité métier propre.
- **Équipe pédagogique de l'École de la Bourse** – acteur indirect, propriétaire du corpus de connaissances et des contenus de formation utilisés par le système.

2.3 Besoins fonctionnels

L'analyse du besoin a conduit à l'identification de dix workflows métier, formalisés comme des routes explicites au sein du graphe d'orchestration (détaillé au chapitre 3). Le tableau 2.1 présente chacun de ces workflows.

Tab. 2.1 : Workflows métier (routes) prises en charge par le système

Route	Besoin fonctionnel couvert
`pedagogie`	Répondre aux questions d'éducation financière (actions, obligations, indices, ordres de bourse, gestion des risques, etc.) à l'aide du corpus documentaire validé (RAG).
`formation_elearning`	Informer sur l'offre de formation et d'e-learning de l'École de la Bourse.
`cisi`	Informer sur la certification professionnelle CISI et les modalités associées.
`quiz`	Proposer des quiz éducatifs adaptés au niveau de l'utilisateur.
`championnat`	Renseigner sur le Championnat de la Bourse et le programme e-Bourse.
`orientation`	Orienter l'utilisateur vers le bon parcours de formation selon son profil et ses objectifs.
`support_ecole`	Fournir un support pratique sur les services de l'École (accès, inscription, identifiants).
`securite`	Intercepter les demandes sensibles (recommandation d'achat/vente, allocation personnalisée) et renvoyer une réponse encadrée, sans jamais exécuter la demande.
`out_of_domain`	Détecter et refuser poliment les demandes hors du périmètre autorisé (éducation financière et services de l'École).

Route	Besoin fonctionnel couvert
`fallback`	Fournir une réponse de secours lorsque la demande n'a pu être rattachée à aucun workflow métier.

À ces besoins « métier » s'ajoutent des besoins fonctionnels transverses :

- traitement de la requête dans la langue de l'utilisateur (français, anglais, arabe standard, darija latine ou arabe) et restitution de la réponse dans la même langue et le même script ;
- continuité conversationnelle au sein d'un même fil de discussion (`thread_id`), permettant des questions de relance (« et quels sont les risques ? ») sans perte de contexte ;
- adaptation du niveau de réponse au profil de l'apprenant (débutant, intermédiaire, avancé) ;
- interaction vocale (question orale, réponse orale) en plus du canal texte ;
- citation systématique des sources documentaires pour toute réponse pédagogique ancrée (RAG) ;
- accessibilité du service via une intégration d'automatisation (n8n) pour de futurs canaux (site web, widget).

2.4 Diagramme de cas d'utilisation

La figure 2.1 synthétise les cas d'utilisation du système du point de vue de l'utilisateur final et des acteurs système.

Le cas *« Requête sensible interceptée »* n'est pas déclenché volontairement par l'utilisateur : il représente le comportement du système lorsqu'une demande de recommandation d'investissement personnalisée est détectée, quelle que soit la route initialement visée. Ce comportement est détaillé à la section 3.3.1.

2.5 Besoins non fonctionnels

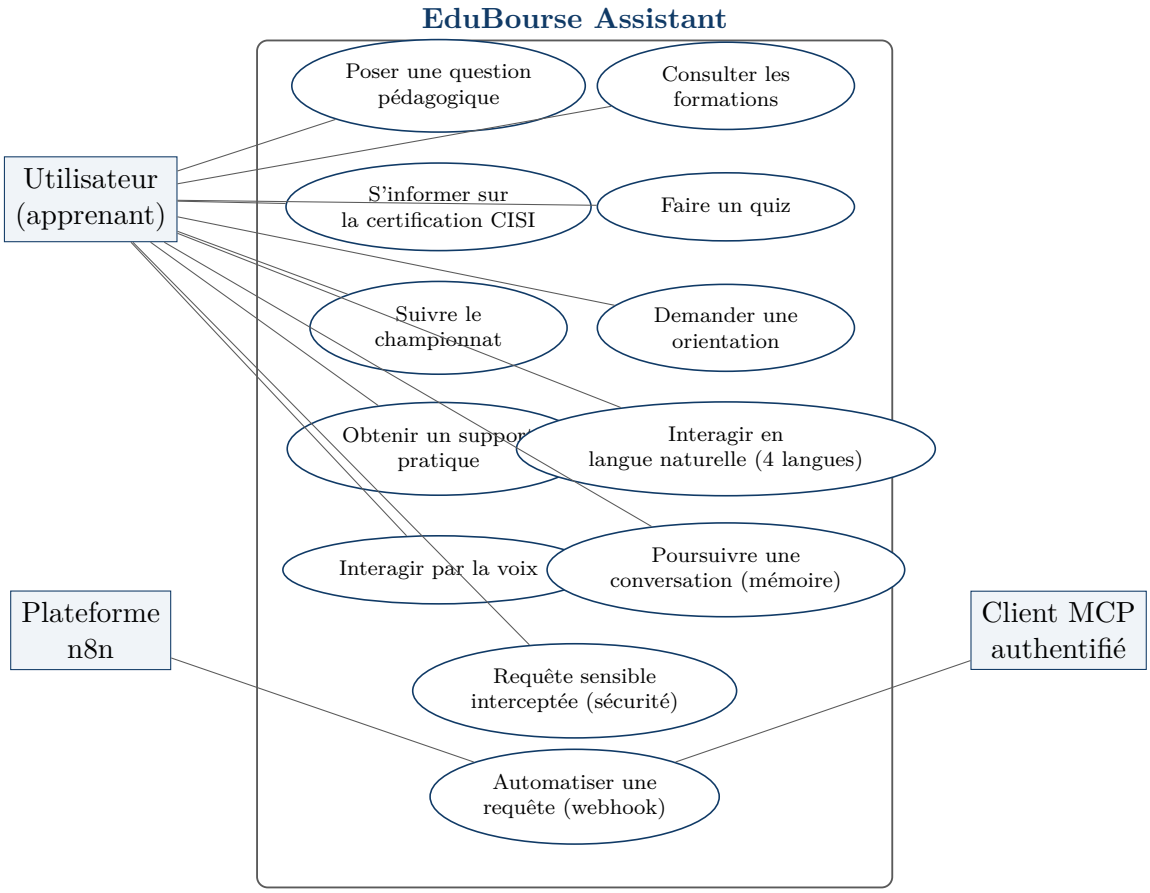


Fig. 2.1 : Diagramme de cas d'utilisation simplifié d'EduBourse Assistant

Tab. 2.2 : Besoins non fonctionnels

Catégorie	Exigence
Sécurité fonctionnelle	Interdiction stricte de toute recommandation d'investissement personnalisée ; échec fermé (*fail-closed*) en cas d'erreur du fournisseur de génération ou de récupération : le système ne doit jamais inventer une réponse financière.
Multilinguisme	Détection et traitement correct de 4 langues métier (français, anglais, arabe standard, darija) et de 2 scripts (latin, arabe) pour la darija.
Traçabilité	Toute réponse pédagogique ancrée doit être accompagnée des identifiants d'évidence et des citations correspondantes.
Disponibilité	Fonctionnement en continu du service local de démonstration (backend, base vectorielle, base relationnelle).
Sobriété des ressources	Fonctionnement sur une machine de démonstration disposant de 8 Go de RAM, avec un seul modèle d'embedding chargé en mémoire (BGE-M3).
Sécurité d'accès	Authentification par jeton pour le serveur MCP, restriction d'hôtes/origines et protection contre le *DNS rebinding*.
Observabilité	Journalisation structurée et bornée des statuts de génération, sans exposition de secrets, de invites (*prompts*) ni de contenu utilisateur brut dans les journaux.
Modularité	Séparation stricte des responsabilités entre le backend (langue, sécurité, mémoire, routage, agents, RAG) et les composants d'intégration (n8n, MCP), qui ne doivent porter aucune logique métier.
Qualité logicielle	Couverture par tests automatisés et analyse statique (*linting*) systématiques sur le backend et le serveur MCP.

2.6 Conclusion du chapitre

L'analyse conduite dans ce chapitre a permis de traduire les objectifs généraux du projet en dix workflows métier explicites, en besoins transverses (multilinguisme, mémoire, voix) et en exigences non fonctionnelles fortes, en particulier sur le plan de la sécurité fonctionnelle. Ces éléments constituent le cahier des charges à partir duquel la conception

présentée au chapitre suivant a été élaborée.

Chapitre 3

Conception

Ce chapitre présente la conception générale et technique d'EduBourse Assistant : l'architecture globale du système, l'organisation interne du graphe multi-agents, les diagrammes UML de séquence et de classes, ainsi que la justification des choix technologiques retenus.

3.1 Architecture générale

Le système est conçu comme un **microservice** interrogé exclusivement via une API REST (`/api/v1/chat`), ce qui permet à plusieurs canaux – interface web, canal vocal, automatisation n8n – de consommer la même logique métier sans la dupliquer. Le backend concentre l'intégralité de l'intelligence applicative : analyse linguistique, politique de sécurité, mémoire conversationnelle, routage, agents spécialisés, récupération documentaire et validation de l'ancrage des réponses. Les composants d'intégration (n8n, serveur MCP, frontend) ne font que transporter et valider la forme des requêtes : ils ne sélectionnent jamais un agent, n'accèdent jamais directement à la base vectorielle ou à la base relationnelle, et ne contournent jamais la politique de sécurité.

La figure 3.1 illustre le parcours d'une requête texte et d'une requête vocale au sein du système.

Trois principes structurent cette architecture :

- **Point de contrôle unique** – toute requête, quel que soit son canal d'origine, traverse la même chaîne *analyse linguistique* → *sécurité* → *mémoire* → *supervision* → *agent*.
- **LangGraph comme routeur autoritaire** – ni n8n ni le serveur MCP ne décident du workflow exécuté ; cette responsabilité appartient exclusivement au graphe d'orchestration.

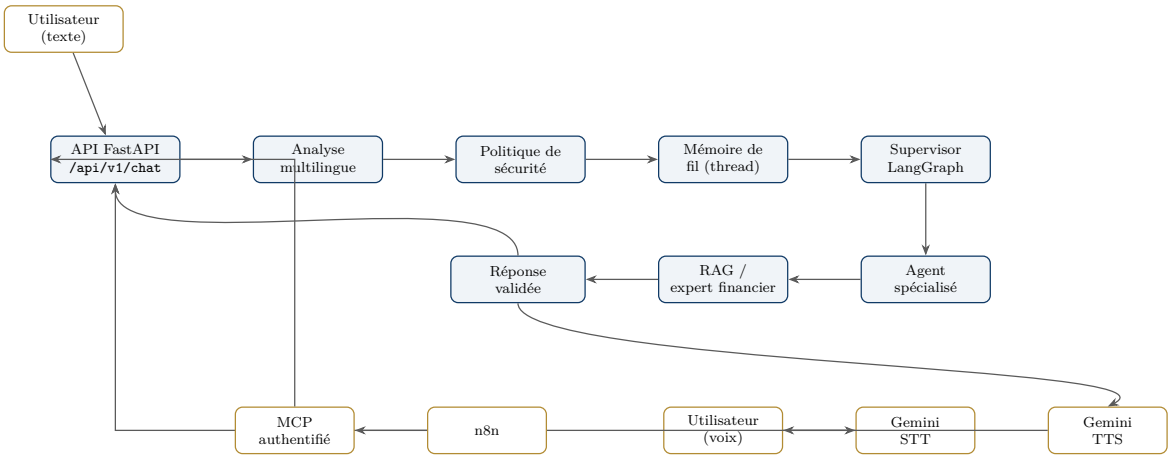


Fig. 3.1 : Architecture générale et parcours des requêtes texte, vocale et automatisée

- **Échec fermé (*fail-closed*)** – en cas d'indisponibilité d'un fournisseur externe (récupération ou génération), le système renvoie un message d'échec maîtrisé plutôt qu'une réponse financière non vérifiée.

3.2 Architecture technique en couches

Le backend est organisé en modules Python fonctionnellement cohérents, résumés dans le tableau 3.1.

Tab. 3.1 : Principaux modules du backend

Module	Responsabilité
`api/`	Endpoints REST (`chat`, `conversations`, `health`, `voice`, `rag`, `rag_admin`, `integrations`) et schémas de requête/-réponse.
`multilingual/`	Détection de langue et de script (français, anglais, arabe, darija latine/arabe), validation de script, normalisation.
`graph/`	Cœur d'orchestration LangGraph : état partagé (`GraphState`), routage, supervision, localisation des réponses, gestion du contexte conversationnel.
`security/` (logique intégrée à `graph/routing.py`)	Détection du domaine autorisé et des demandes sensibles (recommandation personnalisée).
`memory/`	Mémoire conversationnelle, pertinence contextuelle, index vectoriel des échanges passés.

Module	Responsabilité
`persistence/`	Points de reprise (*checkpointer*), historique de conversation, profils d'apprentissage, base relationnelle.
`rag/`	Chargement et audit du corpus, découpage (*chunking*), adaptation de requête, génération ancrée, politique hybride.
`financial_expert/`	Recherche externe approuvée pour les données de marché vérifiables ou datées, avec politique de fraîcheur et de source.
`formation/`, `quiz/`, `orientation/`, `cisi/`, `championnat/`, `support_ecole/`, `out_of_domain/`	Agents métier spécialisés correspondant chacun à un workflow du tableau 2.1.
`voice/`	Fournisseurs de synthèse et reconnaissance vocale (Gemini), gestion audio.
`llm/`	Abstraction des fournisseurs de modèles de langage (Groq).
`observability/`	Journalisation structurée, métriques, mesure de latence.

3.3 Le graphe multi-agents LangGraph

Le cœur du système est un graphe d'états construit avec **LangGraph**, comportant treize nœuds au total : quatre nœuds transverses (analyse linguistique, politique de sécurité, mémoire, supervision/routage) et dix nœuds d'agents métier correspondant aux workflows du tableau 2.1.

L'état partagé du graphe, `GraphState`, est un dictionnaire typé (`TypedDict`) qui circule d'un nœud à l'autre et porte, entre autres : le message utilisateur brut et normalisé, la langue et le script détectés, le statut de domaine et de sécurité, la route sélectionnée et son score de confiance, la réponse métier, les informations d'ancrage RAG (citations, statut de génération, source de connaissance), le profil apprenant, ainsi que les éléments de session (`thread_id`, historique récent). Ce choix d'un état unique, explicite et fortement typé, permet à chaque nœud de ne lire et n'écrire que les champs qui le concernent, tout en gardant une traçabilité complète du cheminement d'une requête.

Le routage vers l'un des dix workflows repose sur une approche **hybride** : une couche lexicale (recherche de termes et de locutions caractéristiques de chaque route, avec replie-

ment Unicode pour tolérer les variantes diacritiques) complétée par des classificateurs **scikit-learn** pré-entraînés et sérialisés (`business_router.joblib` pour le routage métier, `security_intent.joblib` pour la détection d'intention sensible), ce qui permet de combiner la robustesse de règles explicites avec la capacité de généralisation d'un modèle statistique sur des formulations non prévues par les règles.

3.3.1 Politique de sécurité

La politique de sécurité s'articule autour de deux vérifications successives, appliquées avant l'exécution de tout agent :

1. **Restriction de domaine** – la requête doit relever du périmètre autorisé (Bourse, éducation financière, formations et services de l'École) ; à défaut, elle est routée vers `out_of_domain`.
2. **Détection de demande sensible** – la requête est analysée pour détecter une intention de décision personnalisée (achat/vente d'un titre, répartition d'épargne, anticipation de cours), combinant présence de termes financiers, de termes de décision, d'allocation, de dimension temporelle et de tournures personnelles (« pour moi », « dois-je »). Toute requête qualifiée de sensible est interceptée par l'agent `securite`, qui renvoie une réponse encadrée rappelant que le système ne formule ni choix de valeur, ni recommandation de vente, ni répartition financière, et propose à la place des critères généraux d'analyse.

Cette politique est volontairement appliquée *avant* le routage métier : une question par ailleurs pédagogique (« quelle action dois-je acheter ? ») est interceptée dès lors qu'elle exprime une décision personnalisée, indépendamment du fait qu'elle contienne également du vocabulaire boursier légitime.

3.4 Diagramme de séquence – requête pédagogique ancrée (RAG)

La figure 3.2 détaille le déroulement d'une requête relevant du workflow `pedagogie`, du point d'entrée API jusqu'à la génération d'une réponse ancrée dans le corpus documentaire.

Deux points de robustesse méritent d'être soulignés : si la récupération documentaire ne renvoie pas d'évidence suffisante, ou si un fournisseur externe échoue, le pipeline peut basculer vers une recherche financière externe approuvée pour les faits vérifiables et datés, ou renvoyer un message d'échec maîtrisé – jamais une réponse financière inventée.

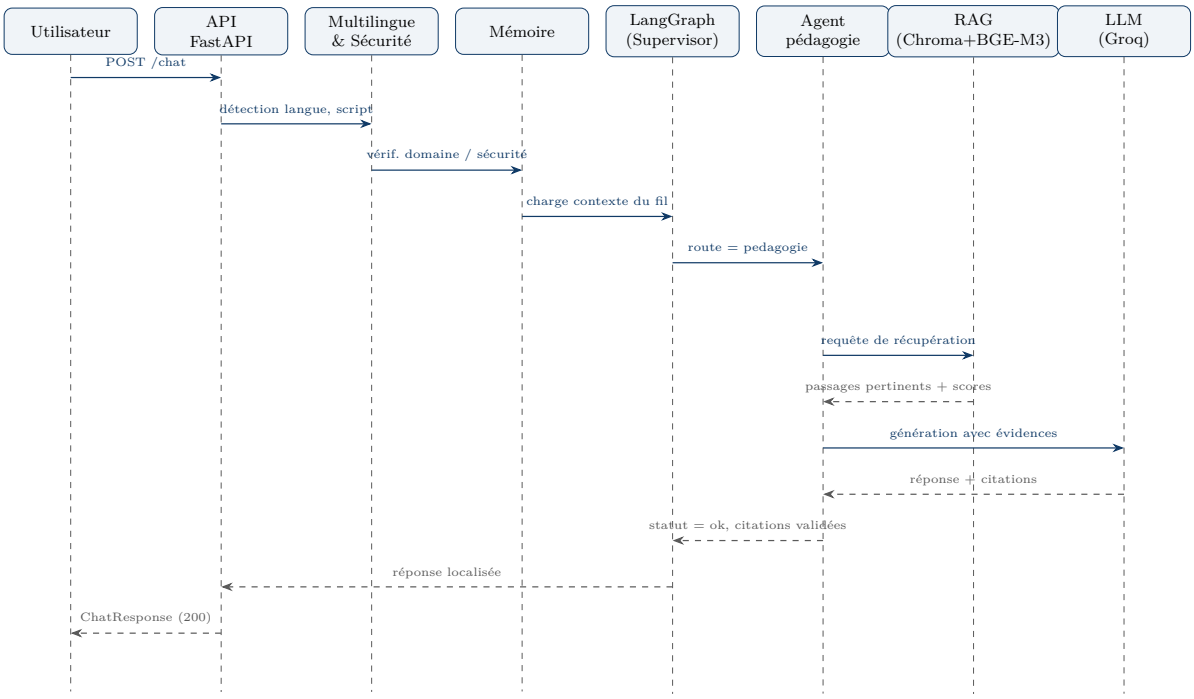


Fig. 3.2 : Diagramme de séquence – traitement d'une requête pédagogique ancrée

3.5 Diagramme de classes simplifié

La figure 3.3 présente les principales structures de données échangées entre les couches API, orchestration et RAG.

`ChatRequest` et `ChatResponse` sont des modèles **Pydantic** en mode strict (`extra="forbid"`), qui rejettent tout champ non prévu et bornent explicitement la taille des champs texte – une garantie de robustesse contre les requêtes malformées ou abusives. `GraphState` n'est pas exposé publiquement : c'est une structure interne au graphe, dont seul un sous-ensemble de champs est projeté vers `ChatResponse` en sortie d'API.

3.6 Choix technologiques et justification

Tab. 3.2 : Choix technologiques et justification

Composant	Technologie	Justification
API backend	FastAPI + Pydantic	Framework asynchrone performant, validation stricte des schémas de données, génération automatique de documentation OpenAPI.

Composant	Technologie	Justification
Orchestration multi-agents	LangGraph (+ LangChain-core)	Modélisation explicite d'un graphe d'états, gestion native des points de reprise (*checkpoints*) pour la continuité conversationnelle.
Base vectorielle	ChromaDB	Base vectorielle légère, adaptée à un déploiement local maîtrisé en ressources, avec filtrage par métadonnées.
Modèle d'embedding	BGE-M3 (1024 dimensions)	Modèle multilingue robuste sur le français, l'anglais et l'arabe, cohérent avec les quatre langues métier du projet.
Classification légère	scikit-learn (+ joblib)	Classificateurs de routage métier, de détection d'intention sensible et de détection de langue, complémentaires à l'approche par règles lexicales.
Inférence LLM (génération)	Groq	Latence d'inférence réduite, adaptée à un usage conversationnel interactif.
Voix & localisation	Google Gemini (STT/TTS)	Couverture multilingue de la reconnaissance et de la synthèse vocale, alternative retenue après exclusion d'ElevenLabs pour restriction du palier gratuit.
Base relationnelle	PostgreSQL + SQLAlchemy + Alembic	Persistance applicative (historique, profils apprenants), migrations versionnées.
Intégration/Automatisation	n8n	Plateforme d'automatisation open-source pour l'orchestration de webhooks externes sans logique métier propre.
Interface d'outils IA	MCP (*Model Context Protocol*)	Standard ouvert d'exposition d'un ensemble d'outils restreint et authentifié à un client automatisé.
Frontend de démonstration	React 19 + TypeScript + Vite	Développement rapide d'un tableau de bord typé, multi-pages, avec un existant riche en composants réutilisables.
Conteneurisation	Docker / Docker Compose	Isolation et reproductibilité des services (backend, PostgreSQL, ChromaDB, MCP, n8n).

Composant	Technologie	Justification
Outillage Python	`uv`, `ruff`, `pytest`	Gestion rapide des dépendances, analyse statique et tests automatisés intégrés au flux de développement.

3.7 Conclusion du chapitre

La conception retenue répond directement aux besoins identifiés au chapitre 2 : un point de contrôle unique pour la sécurité et le routage, un état de graphe explicite et traçable, une génération de réponses systématiquement ancrée et citée, et une séparation stricte entre logique métier (backend) et transport (n8n, MCP). Le chapitre suivant détaille la réalisation concrète de chacun de ces éléments.

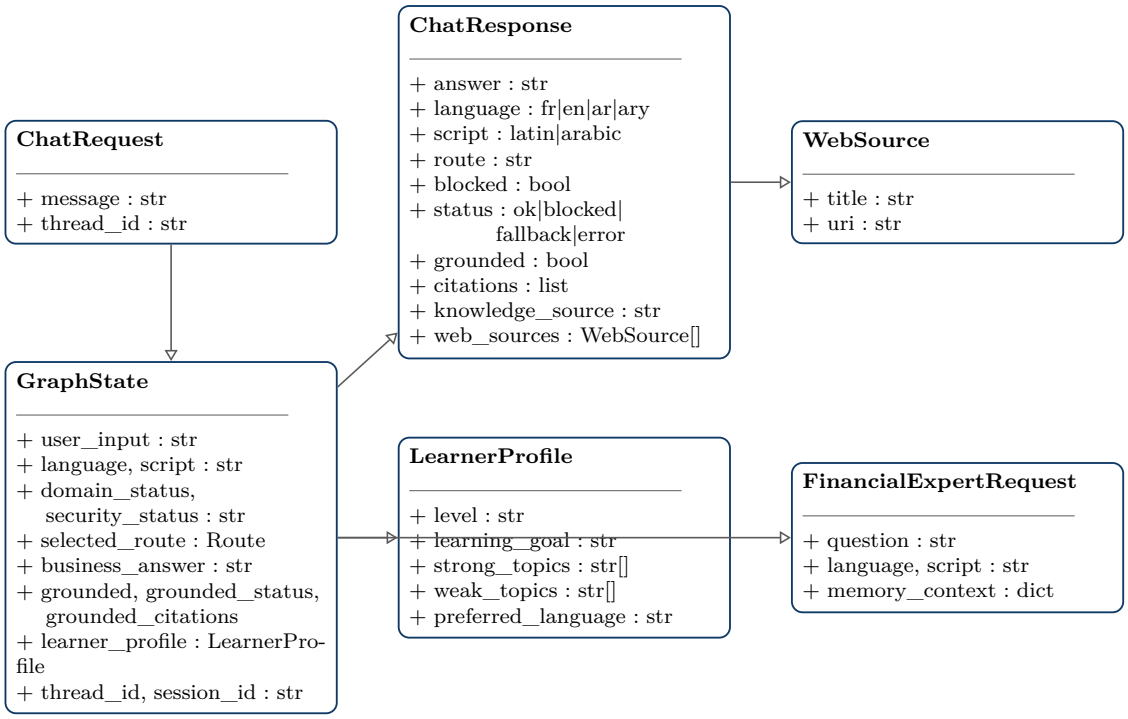


Fig. 3.3 : Diagramme de classes simplifié – objets d'échange API / graphe / RAG

Chapitre 4

Réalisation

Ce chapitre décrit la mise en œuvre concrète des composants conçus au chapitre 3 : environnement de travail, pipeline multilingue, sécurité, récupération documentaire, agents métier, mémoire conversationnelle, canal vocal, intégrations externes et interface de démonstration.

4.1 Environnement de travail

Le backend est développé en **Python 3.11+** avec **FastAPI** pour l'API REST et **LangGraph** pour l'orchestration. La gestion des dépendances est assurée par `uv`, avec verrouillage explicite des versions (`uv.lock`) et export reproductible vers `requirements.txt`. La qualité du code est vérifiée par `ruff` (analyse statique) et la suite de tests par `pytest`, exécutés systématiquement avant toute stabilisation de jalon. L'ensemble des services (backend, PostgreSQL, ChromaDB, serveur MCP, n8n) est orchestré par **Docker Compose**, avec des réseaux Docker externes dédiés pour isoler les flux MCP $\leftrightarrow$ backend et n8n $\leftrightarrow$ backend/MCP. Le frontend de démonstration est développé en **React 19 / TypeScript**, avec **Vite** comme outil de build.

Le développement a été mené sur une machine de démonstration contrainte à 8 Go de RAM, ce qui a directement influencé plusieurs choix d'implémentation : un seul modèle d'embedding (BGE-M3) chargé en mémoire, persistance conversationnelle désactivée par défaut (`PERSISTENCE_ENABLED=false`), et arrêt des conteneurs non indispensables (frontend, pgAdmin) pendant les phases de test intensif.

4.2 Pipeline de traitement multilingue

Le module `multilingual/` assure la détection de la langue et du script du message entrant, étape préalable à tout routage. Il combine :

- un **détecteur statistique** (`statistical_detector.py`) reposant sur des indices lexicaux et n-grammes ;
- un **classificateur scikit-learn** sérialisé (`language_detector.joblib`), accompagné de ses métriques d'évaluation, pour trancher les cas ambigus ;
- un **détecteur de script** (`script_detector.py`) distinguant graphie latine et graphie arabe, essentiel pour la darija qui s'écrit dans les deux systèmes ;
- un **validateur de script** (`script_validator.py`) qui vérifie, en sortie, que la réponse générée respecte bien le script attendu par l'utilisateur.

Les langues métier normalisées sont `fr`, `en`, `ar` et `ary` (darija), cette dernière recouvrant deux étiquettes internes de détection, `ary-arabic` et `ary-latin`, harmonisées en une unique langue métier `ary` par la fonction `normalize_business_language`. Ce choix de normalisation a simplifié le reste du pipeline : le routage, la génération et la personnalisation raisonnent sur quatre langues métier stables, tandis que le script (latin ou arabe) est traité comme une dimension orthogonale portée séparément dans l'état du graphe.

Des expérimentations complémentaires (évaluation sur le corpus multilingue **FLORES-200**, jeux de diagnostic dédiés à l'arabe et à la darija) ont été menées en amont pour arbitrer les choix de seuils et de features des détecteurs, en particulier pour la darija, langue à faibles ressources dont la couverture par les outils de traitement automatique du langage reste hétérogène.

4.3 Sécurité et conformité

La politique de sécurité, décrite en conception à la section 3.3.1, est implémentée principalement dans `graph/routing.py`. La fonction `is_in_domain` détermine si la requête relève du périmètre autorisé, à partir d'un dictionnaire de termes et de locutions métier couvrant les quatre langues (y compris les graphies arabe et darija). La fonction `is_sensitive_request` combine plusieurs familles d'indices lexicaux – actifs financiers, verbes de décision, termes d'allocation, marqueurs temporels, tournures personnelles – pour détecter une demande de recommandation personnalisée, tout en évitant les faux positifs sur les questions purement éducatives (« qu'est-ce qu'une action ? »).

Listing 4.1 : Extrait simplifié de la détection de demande sensible (`graph/routing.py`)

```
def is_sensitive_request(text: str, language: str = "unknown") -> bool:
    tokens = tokenize(text)
    token_set = set(tokens)
    assets = bool(token_set & FINANCIAL_ASSET_TERMS)
    decision = bool(token_set & DECISION_TERMS)
```

```
    allocation = bool(token_set & ALLOCATION_TERMS)
    ...
    decision_request = (
        (assets and (decision or explicit)) or actionable_without_asset
    ) and not educational
    return allocation_request or decision_request or timing_request or
        imperative_request
```

Lorsqu'une requête est jugée sensible, l'agent `securite` court-circuite tout traitement métier ultérieur et renvoie une réponse fixe, encadrée, rappelant les limites du système. Cette interception a été validée par une matrice de tests dédiée, couvrant les quatre langues et les deux canaux (texte, voix) – cf. chapitre 5.

Au niveau infrastructure, le serveur MCP applique une authentification par jeton statique local, une liste explicite d'hôtes et d'origines autorisés, et une protection contre le *DNS rebinding*. Aucune valeur secrète (jetons, identifiants) n'est jamais recopiée dans la documentation ou les journaux applicatifs.

4.4 Récupération documentaire et génération ancrée (RAG)

Le pipeline RAG repose sur les étapes suivantes :

1. **Constitution et audit du corpus** (`corpus_audit.py`, `text_extraction.py`) : extraction et contrôle qualité des documents source approuvés par l'École de la Bourse.
2. **Découpage** (`chunking.py`) : segmentation des documents en passages indexables.
3. **Indexation vectorielle** (`chroma_index.py`) : encodage des passages avec le modèle **BGE-M3** (1024 dimensions) et stockage dans la collection **ChromaDB** figée `j6_bge_m3_content_v1`, comptant 353 embeddings.
4. **Adaptation de requête** (`query_adapter.py`) : reformulation éventuelle de la requête de récupération selon la langue et le contexte conversationnel, sans jamais modifier le routage métier ni la langue de restitution.
5. **Récupération** (`retrieval_service.py`) : sélection des passages les plus pertinents selon une politique hybride (`hybrid_policy.py`) combinant similarité vectorielle et filtres de domaine (`domain_policy.py`).
6. **Génération ancrée** (`grounded_generation.py`) : production de la réponse par le LLM à partir des seules évidences récupérées, avec validation stricte des identifiants d'évidence, des citations et du script de sortie.

La génération ancrée renvoie systématiquement l'un des statuts canoniques suivants : `ok`, `retrieval_provider_error`, `retrieval_configuration_error`, `generation_provider_error`, `generation_configuration_error`, `insufficient_evidence` ou `grounding_error`. Seul le statut `ok`, accompagné d'une réponse non vide, est considéré comme pleinement ancré ; tout autre statut déclenche soit un repli vers la recherche financière externe approuvée (pour les faits de marché vérifiables), soit un message d'échec maîtrisé – jamais une réponse inventée.

4.5 Agents métier spécialisés

Chaque route du tableau 2.1 est implémentée par une fonction d'agent dédiée dans `graph/agents.py`, orchestrée par le graphe LangGraph. Le tableau 4.1 résume le comportement de chacune.

Tab. 4.1 : Comportement des agents métier

Agent	Comportement
`pedagogie_agent`	Appelle le service de génération ancrée ; si l'évidence est insuffisante ou si la question porte sur une donnée de marché vérifiable et datée, bascule vers la recherche financière externe approuvée (`FinancialExpertService`).
`formation_agent`	Oriente vers le flux d'évaluation de niveau et de recommandation de formation, ou délègue au flux légacy selon l'état de session déjà présent.
`cisi_agent`	Fournit l'information relative à la certification CISI ; détecte une éventuelle intention de formation e-learning associée.
`quiz_agent`	Génère un quiz adapté, en s'appuyant si disponible sur le service de profil apprenant.
`championnat_agent`	Fournit l'information relative au Championnat de la Bourse.
`orientation_agent`	Recommande un parcours de formation à partir du profil et des objectifs déclarés.
`support_ecole_agent`	Répond aux questions pratiques sur les services de l'École (accès, inscription).
`securite_agent`	Intercepte toute requête sensible et renvoie une réponse encadrée fixe (cf. section 3.3.1).
`out_of_domain_agent`	Renvoie une réponse de refus/redirection pour toute requête hors périmètre.

Agent	Comportement
`fallback_agent`	Réponse de secours lorsqu'aucune route n'a pu être déterminée avec confiance.

4.6 Mémoire conversationnelle et personnalisation

La continuité d'une conversation repose sur un **point de reprise** (*checkpointer*) LangGraph identifié par un `thread_id` fourni par le client. En configuration de démonstration, le checkpointer utilisé est `MemorySaver` : la continuité est assurée tant que le processus backend reste actif, mais n'est pas durable après un redémarrage (`PERSISTENCE_ENABLED=false`). Une architecture de persistance PostgreSQL est prévue en infrastructure (`persistence/`) pour une évolution future vers une continuité durable, sous réserve de résoudre le chargement d'une seconde instance du modèle d'embedding en mémoire.

Au-delà du simple historique, un service de **mémoire pertinente** (`relevant_memory_service.py`) identifie, parmi les échanges passés d'un même fil, ceux qui sont sémantiquement pertinents pour la question courante, à l'aide d'un index vectoriel dédié (`vector_index.py`, `embeddings.py`). Cette fonctionnalité est protégée par un indicateur de fonctionnalité (*feature flag*) afin de préserver l'intégrité des campagnes de test qui ne souhaitent pas de mémoire inter-tours.

La personnalisation repose sur un **profil apprenant** (`learner_profile`) à trois niveaux – débutant, intermédiaire, avancé – utilisé notamment par les agents `quiz` et `orientation` pour adapter la difficulté et les recommandations.

4.7 Interface vocale

Le canal vocal réutilise intégralement le pipeline texte : la voix de l'utilisateur est transcrite par **Gemini STT**, le texte obtenu est traité par la même chaîne *analyse* → *sécurité* → *mémoire* → *supervision* → *agent*, puis la réponse textuelle est restituée sous forme audio par **Gemini TTS**. Le module `voice/` encapsule la gestion du format audio (`wav.py`, `audio.py`), la normalisation des erreurs fournisseur (`errors.py`) et l'abstraction des fournisseurs (`providers/gemini.py`, `providers/live.py` pour un mode conversationnel en flux continu). Ce choix garantit qu'aucune logique métier ou de sécurité n'est dupliquée pour le canal vocal.

4.8 Intégration n8n et serveur MCP

L'intégration avec la plateforme d'automatisation **n8n** illustre le principe de point de contrôle unique : le workflow n8n ne valide que la forme de la requête (présence de `message` et de `thread_id`) avant de la transmettre au serveur **MCP**, qui l'achemine à son tour vers `POST /api/v1/chat`. Le serveur MCP expose exactement deux outils (`get_backend_health`, `ask_edubourse`) et deux ressources (`edubourse://health`, `edubourse://capabilities`) ; il ne dispose d'aucune capacité directe d'accès à la base vectorielle, à la base relationnelle, au système de fichiers ou à un interpréteur de commandes. La réutilisation de réseaux Docker dédiés entre n8n, MCP et le backend constitue une simple plomberie de transport et ne confère, par elle-même, aucune capacité métier supplémentaire à ces composants.

4.9 Interface de démonstration (frontend)

Le frontend, développé en React 19/TypeScript avec Vite, propose un tableau de bord sombre à l'esthétique inspirée de Dribbble, organisé en pages thématiques correspondant à chaque sous-système : *Assistant* (conversation en direct), *Agents* (catalogue et orchestration des agents), *Architecture* (visualisation du pipeline), *Multilingue* (testeur de détection de langue et de voix), *RAG* (recherche sémantique et sources), *Sécurité* (testeur de politique de sécurité), *Validation* (tableau de bord de résultats de tests), *Voix* (contrôle de session vocale) et *Intégrations* (tests des intégrations n8n/MCP). Cette interface a vocation de démonstration et de pilotage technique ; elle respecte le contrat d'intégration défini en conception : elle n'appelle jamais directement ChromaDB ou PostgreSQL et ne fait que consommer l'API REST du backend.

[Espace réservé pour l'insertion de captures d'écran de l'interface – pages Assistant, Agents et Architecture recommandées en priorité.]

4.10 Conclusion du chapitre

Ce chapitre a détaillé la réalisation de l'ensemble des composants du système : traitement multilingue hybride, politique de sécurité à deux niveaux, pipeline RAG ancré et traçable, dix agents métier spécialisés, mémoire conversationnelle personnalisée, canal vocal et intégrations externes respectant le principe de point de contrôle unique. Le chapitre suivant présente la stratégie de tests mise en œuvre pour valider l'ensemble de ces réalisations, ainsi que les résultats obtenus.

Chapitre 5

Tests, validation et résultats

Ce chapitre présente la stratégie de tests adoptée tout au long du projet, les résultats de validation obtenus lors de l'audit technique final, les limitations connues du système ainsi que le scénario de démonstration utilisé pour la soutenance.

5.1 Stratégie de tests

La qualité du système a été assurée par une double discipline de tests automatisés et d'analyse statique, exécutée à chaque jalon de développement :

- des **tests unitaires et d'intégration** `pytest` côté backend, couvrant la détection linguistique, le routage, la politique de sécurité, le pipeline RAG, les agents métier, la mémoire conversationnelle et les endpoints API ;
- des **tests dédiés** au serveur MCP, vérifiant l'authentification, la restriction d'hôtes/origines et le comportement des deux outils exposés ;
- une **analyse statique** `ruff` sur le backend et le serveur MCP, exécutée systématiquement avant toute stabilisation de jalon ;
- des **scénarios de validation manuelle multicanal** (texte et voix) pour les quatre langues métier, nécessitant un client Python en UTF-8 pour fiabiliser les tests arabe et darija (un terminal PowerShell mal configuré pouvant produire de faux problèmes d'encodage, à ne pas confondre avec une défaillance de localisation du produit).

5.2 Résultats de l'audit technique final

Le tableau 5.1 synthétise les résultats obtenus lors de la campagne de validation finale (jalons `J9-01` à `J9-05`).

Tab. 5.1 : Résultats de la validation finale

Indicateur	Résultat
Tests backend (pytest)	1 181 passés, 0 échoué, 0 erreur, 0 ignoré
Tests serveur MCP	14 passés
Analyse statique (ruff) backend & MCP	Conforme (*PASS*)
Validation multilingue	Français, anglais, arabe standard, darija (latin et arabe) :
Matrice de sécurité	Conforme (*PASS*)
Intégrité du corpus RAG	Conforme – collection `j6_bge_m3_content_v1`, 353 embed

Ces résultats couvrent l'ensemble des quatre langues métier et les deux canaux (texte et voix), avec une vérification explicite qu'aucune réponse à caractère financier n'est délivrée aux utilisateurs sans passer par la politique de sécurité.

5.3 Validation du comportement en situation d'échec

Une attention spécifique a été portée à la validation du comportement du système en cas d'échec d'un fournisseur externe (récupération ou génération). Le principe d'**échec fermé** (*fail-closed*) a été vérifié : en cas d'erreur transitoire de génération (`generation_provider_error`), le système renvoie une réponse d'échec maîtrisée plutôt qu'une réponse inventée, et la procédure de récupération recommandée – ouverture d'un nouveau fil de discussion puis nouvelle tentative – a été documentée et testée, sans jamais nécessiter le redémarrage de l'ensemble de la plateforme pour un incident isolé.

5.4 Limitations connues

L'audit final a documenté explicitement sept limitations opérationnelles, résumées dans le tableau 5.2. Ces limitations sont considérées comme des contraintes de configuration de l'environnement de démonstration, et non comme des défauts bloquants du produit.

Tab. 5.2 : Limitations connues documentées à l'issue du projet

N°	Limitation
1	La persistance est désactivée (`PERSISTENCE_ENABLED=false`) et le *checkpointer* utilisé est `MemorySaver`.
2	La continuité d'un même fil de discussion n'est pas durable après un redémarrage du backend.
3	L'ancrage fourni par les fournisseurs externes peut connaître une variabilité transitoire.

N°	Limitation
4	L'image backend en conteneur peut ponctuellement accuser un décalage par rapport au code source lorsqu'une reconstruction hors ligne n'est pas disponible ; la parité d'exécution validée est suivie séparément.
5	La base SQLite de n8n comporte neuf lignes orphelines liées à un workflow supprimé (statistiques, exécutions, informations d'*insights*), sans impact sur l'intégrité fonctionnelle (cinq workflows, deux identifiants et 121 exécutions intacts).
6	Le serveur MCP utilise un jeton de service statique unique, représentant un seul domaine de confiance plutôt qu'une isolation d'identité par client.
7	La machine de démonstration dispose de 8 Go de RAM, avec environ 3,8 Gio disponibles pour Docker, ce qui contraint le nombre de services actifs simultanément.

5.5 Scénario de démonstration

Le scénario de démonstration utilisé pour valider le système de bout en bout suit la séquence suivante :

1. vérification de l'état de santé du backend, de n8n et du serveur MCP ;
2. question pédagogique en français (« C'est quoi une action ? ») avec vérification de la route `pedagogie`, de l'ancrage et des citations ;
3. question d'orientation (« Quelle formation choisir pour débuter ? ») ;
4. génération d'un quiz (« Fais-moi un quiz sur la Bourse. ») ;
5. question sur le Championnat de la Bourse ;
6. question sensible (« Quelle action dois-je acheter ? ») avec vérification de l'interception par la politique de sécurité ;
7. question hors domaine (« Quelle est la capitale du Japon ? ») avec vérification du refus/redirection ;
8. questions en arabe standard et en darija (graphie arabe), avec un client Python UTF-8 ;
9. question en darija latine (« Chno hiya l'action ? ») ;
10. question de relance dans un même fil (« Et quels sont les risques ? ») avec vérifi-

cation de la continuité du `thread_id` et des citations ;

11. requête identique transmise via le webhook n8n vers le serveur MCP, démontrant la réutilisation de la même politique de sécurité et de routage.

En cas d'échec transitoire d'ancrage durant la démonstration, la procédure prévoit de conserver les services actifs, d'ouvrir un nouveau fil de discussion et de retenter une seule fois, sans jamais substituer manuellement une réponse à l'échec constaté.

5.6 Conclusion du chapitre

La campagne de validation finale confirme que le système répond aux exigences fonctionnelles et non fonctionnelles définies au chapitre 2 : couverture complète des quatre langues métier, politique de sécurité vérifiée, ancrage documentaire contrôlé, et robustesse démontrée par plus de 1 190 tests automatisés passants. Les limitations identifiées sont documentées, circonscrites et sans impact sur le périmètre fonctionnel validé pour la soutenance.

Conclusion générale et perspectives

Ce Projet de Fin d'Année, mené au sein de la Bourse de Casablanca pour son École de la Bourse, avait pour objectif de concevoir et de développer un assistant conversationnel intelligent, multilingue et sécurisé, capable d'accompagner le grand public dans l'apprentissage des concepts boursiers et financiers.

La démarche suivie a permis d'aboutir à un système complet : une architecture en microservice orchestrée par un graphe multi-agents LangGraph de treize nœuds, un pipeline de génération augmentée par récupération ancré sur un corpus documentaire vectorisé et systématiquement cité, un traitement hybride (règles lexicales et classificateurs statistiques) de quatre langues métier – dont la darija marocaine en deux graphies –, ainsi qu'une politique de sécurité fonctionnelle stricte interdisant toute recommandation d'investissement personnalisée. Le système intègre également un canal vocal complet, une mémoire conversationnelle personnalisée, et une intégration à la plateforme d'automatisation n8n via un serveur MCP authentifié à surface d'action volontairement restreinte. L'ensemble a été validé par plus de 1 190 tests automatisés passants et par une matrice de validation multilingue et sécuritaire couvrant tous les canaux supportés.

Sur le plan personnel, ce projet a constitué une mise en pratique concrète et exigeante de compétences couvrant l'ensemble de la chaîne d'un système d'intelligence artificielle appliqué : conception d'architectures multi-agents, traitement automatique de langues à ressources inégales, ingénierie de systèmes RAG fiables, conception d'API robustes, et intégration de contraintes de sécurité métier dès la phase de conception plutôt qu'en correction a posteriori. Il a également nécessité une gestion rigoureuse des compromis entre ambition fonctionnelle et contraintes réelles (quotas des fournisseurs de modèles, ressources machine limitées, délai du projet), formalisée par la démarche méthodologique itérative présentée au chapitre 1.

Perspectives d'évolution

Plusieurs axes d'évolution ont été identifiés à l'issue du projet, certains déjà documentés comme limitations connues (chapitre 5) :

- activer une **persistance durable** de la mémoire conversationnelle via un *checkpointer* PostgreSQL, sous réserve de résoudre la contrainte mémoire liée au chargement d'une seconde instance du modèle d'embedding ;
- renforcer l'**isolation d'identité** du serveur MCP par client, au-delà du jeton de service statique unique actuel ;
- élargir et approfondir le **corpus documentaire** validé, en particulier pour la darija, afin de réduire le recours à la recherche financière externe ;
- industrialiser le **pipeline de reconstruction** de l'image backend afin de supprimer tout risque de décalage entre le code source et l'image déployée ;
- déployer l'interface de démonstration en **production**, au-delà du périmètre actuel de tableau de bord technique, avec une expérience utilisateur dédiée au grand public ;
- explorer une **évaluation continue** de la qualité de génération (au-delà des tests fonctionnels), par exemple via des jeux d'évaluation humaine réguliers sur les quatre langues.

Ce projet démontre qu'il est possible de concilier richesse conversationnelle et exigence réglementaire dans un contexte financier sensible, à condition de faire de la sécurité et de la traçabilité des réponses des contraintes de conception de premier ordre, et non des ajouts périphériques. Il constitue une base solide pour un futur déploiement au service de la mission d'éducation financière de l'École de la Bourse.

Bibliographie

[1] Wikipédia, *Bourse des valeurs de Casablanca*. https://fr.wikipedia.org/wiki/Bourse_des_valeurs_de_Casablanca (consulté en septembre 2026).

[2] Bourse de Casablanca, *Se former pour comprendre la Bourse – École de la Bourse*. https://www.casablanca-bourse.com/fr/se-former-pour-comprendre-la-bourse (consulté en septembre 2026).

[3] École de la Bourse, *Site officiel*. https://www.ecole-bourse.ma/ (consulté en septembre 2026).

[4] FastAPI, *Documentation officielle*. https://fastapi.tiangolo.com/.

[5] LangChain AI, *LangGraph – Documentation officielle*. https://langchain-ai.github.io/langgraph/.

[6] Chroma, *Documentation officielle*. https://docs.trychroma.com/.

[7] Chen, J. et al., *BGE M3-Embedding : Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*, 2024. https://arxiv.org/abs/2402.03216.

[8] Anthropic, *Model Context Protocol – Spécification*. https://modelcontextprotocol.io/.

[9] n8n, *Documentation officielle*. https://docs.n8n.io/.

[10] CISI (Chartered Institute for Securities & Investment), *Site officiel*. https://www.cisi.org/.

Annexe A

Exemple de requête et de réponse de l'API

Listing A.1 : Requête HTTP vers l'API de chat

```
POST /api/v1/chat HTTP/1.1
Content-Type: application/json

{
  "message": "C'est quoi une action en Bourse ?",
  "thread_id": "demo-fr-001"
}
```

Listing A.2 : Réponse HTTP correspondante (extrait)

```
{
  "answer": "Une action represente une part du capital d'une societe...",
  "language": "fr",
  "script": "latin",
  "route": "pedagogie",
  "blocked": false,
  "thread_id": "demo-fr-001",
  "status": "ok",
  "grounded": true,
  "knowledge_source": "rag",
  "citations": [ { "...": "..." } ]
}
```

Annexe B

Variables de configuration principales

Le fichier `.env.example` documente, sans jamais en fournir les valeurs réelles, les variables de configuration nécessaires au déploiement local : paramètres de connexion à la base de données PostgreSQL, clé d'API du fournisseur d'inférence (`GROQ_API_KEY`), identifiants du fournisseur Gemini (voix et localisation), jeton d'authentification du serveur MCP (`EDUBOURSE_MCP_AUTH_TOKEN`), jeton interne de l'intégration n8n (`EDUBOURSE_N8N_INTERNAL_TO`) ainsi que les URL des différents services. Conformément à la politique de sécurité du projet, aucune valeur de ce fichier n'est jamais recopiée dans la documentation, les journaux applicatifs ou le présent rapport.

Annexe C

Statuts canoniques de génération ancrée

Statut	Signification
`ok`	Réponse générée et ancrée avec succès dans le corpus approuv
`retrieval_provider_error`	Échec du fournisseur de récupération (base vectorielle).
`retrieval_configuration_error`	Erreur de configuration de la récupération.
`generation_provider_error`	Échec du fournisseur de génération (LLM).
`generation_configuration_error`	Erreur de configuration de la génération.
`insufficient_evidence`	Évidence documentaire insuffisante pour répondre de façon an
`grounding_error`	Échec de validation de l'ancrage (citations, script, évidence).

Annexe D

Journal des jalons de développement

Le journal de développement du projet (`DEVELOPMENT_STATUS.md`) trace l'avancement par jalon, avec des statuts intermédiaires explicites :

- **J3** – noyau LangGraph, infrastructure de supervision et de fournisseurs, sécurité, métadonnées multilingues, gestion des points de reprise et des limites de débit ; suite de 320 tests ; statut `STABLE_PROVISIONAL` dans l'attente d'un audit indépendant final.
- **J4** – pipeline texte-à-texte complet, API v1, intégration multilingue, sécurité, LangGraph, agents, localisation, validation de script et documentation OpenAPI ; statut `STABLE_PROVISIONAL`, test de fumée avec fournisseurs réels et acceptation système finale alors en attente.
- **J9-01 à J9-05** – audit technique final : 1 181 tests backend passés, 14 tests MCP passés, analyse statique conforme, validation multilingue et sécuritaire conforme, intégrité du corpus RAG confirmée. Statut final : système prêt pour l'audit de projet et la démonstration encadrée de soutenance.